Assignment-13.5

Name:k.likhith
Rollno:2303A51331
Batch-20
Task-1: Refactoring – Removing Global Variables

```python
def calculate_interest(amount, rate):
    return amount * rate

amount = float(input("Enter amount: "))
rate = float(input("Enter interest rate: "))

interest = calculate_interest(amount, rate)

print("Interest:", interest)
```

Output:

```
Enter amount: 12000
Enter interest rate: 2
Interest: 24000.0
```

Observation: The program was refactored to remove the global variable and pass the value as a function parameter.This improves modularity, flexibility, and makes the function easier to test.

Task2: Refactoring Deeply Nested Conditionals

```python
score = int(input("Enter score: "))

if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

Output:

```
Enter score: 75
Very Good
```

Observation: The deeply nested if–else statements were replaced with a cleaner if-elif-else structure.This improves readability and simplifies the program logic.

Task3: Refactoring File Handling

```python
def read_file(filename):
    try:
        with open(filename) as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")

file1 = input("Enter first file name: ")
file2 = input("Enter second file name: ")

read_file(file1)
read_file(file2)
```

Output:

```
nter first file name: simple.txt
nter second file name: sample.txt
ile 'simple.txt' not found.
ile 'sample.txt' not found.
```

Observation: Repeated file handling code was replaced with a reusable function using with open().This follows the DRY principle and ensures files are safely closed.

Task-4: Optimized Search Logic

```python
users = {"admin", "guest", "editor", "viewer"}

name = input("Enter username: ")

print("Access Granted" if name in users else "Access Denied")
```

Output:

```
Enter username: guest
Access Granted
```

Observation: The list-based search was replaced with a set for faster membership checking. This improves the time complexity from **O(n) to O(1)**.

Task-5: OOP Design

```python
class EmployeeSalaryCalculator:

    def __init__(self, salary):
        self.salary = salary
        self.tax_rate = 0.2

    def calculate_tax(self):
        return self.salary * self.tax_rate

    def net_salary(self):
        return self.salary - self.calculate_tax()

salary = float(input("Enter salary: "))

emp = EmployeeSalaryCalculator(salary)

print("Net Salary:", emp.net_salary())
```

Output:

```
Enter salary: 500000
Net Salary: 400000.0
```

Observation: The procedural code was converted into a class-based design using OOP principles.This improves encapsulation and makes the code more organized and reusable.

Task-6: Performance Optimization

```python
n = int(input("Enter range limit: "))

total = sum(i for i in range(1, n) if i % 2 == 0)

print("Sum of even numbers:", total)
```

Output:

```
Enter range limit: 5
Sum of even numbers: 6
```

Observation: The loop-based summation was optimized using Python's built-in sum() with a generator expression.This reduces code complexity and improves execution efficiency.

Task-7: Removing Hidden Side Effects

```python
def add_item(data, x):
    new_data = data.copy()
    new_data.append(x)
    return new_data

data = []

n = int(input("How many items to add: "))

for i in range(n):
    value = int(input("Enter value: "))
    data = add_item(data, value)

print("Final list:", data)
```

Output:

```
How many items to add: 3
Enter value: 101
Enter value: 102
Enter value: 108
Final list: [101, 102, 108]
```

Observation: The function was modified to return a new list instead of modifying a global variable.This avoids hidden side effects and makes the function behavior predictable.

Task-8: Password Validation

```python
def is_length_valid(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

password = input("Enter password: ")

if not is_length_valid(password):
    print("Password too short")
elif not has_digit(password):
    print("Must contain digit")
elif not has_uppercase(password):
    print("Must contain uppercase")
else:
    print("Valid Password")
```

Output:

```
Enter password: Api86#
Password too short
```

Observation: Complex nested validation logic was divided into small validation functions. This improves readability, modularity, and makes the code easier to test.